

Introduction to Variables

A variable is like a box/container that can HOLD something inside it.

Inside the box, anytime you can change it.

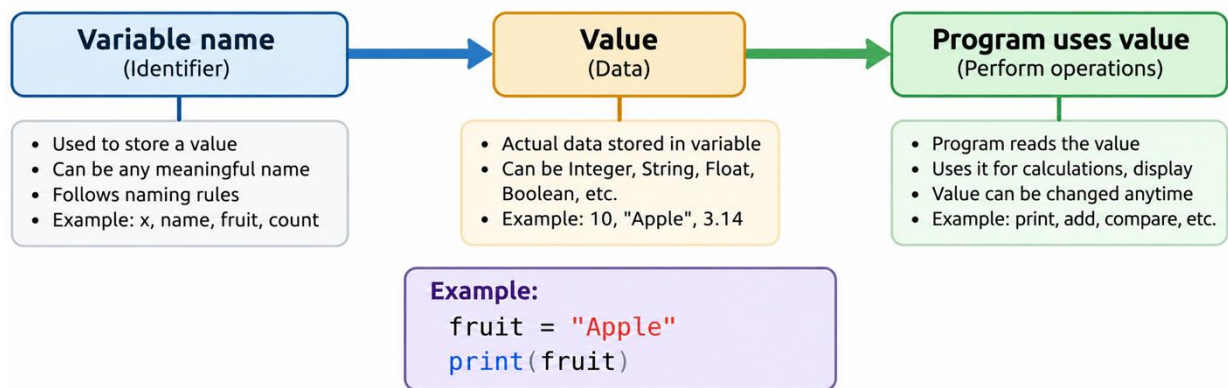
The value can be changed during program execution.

Python auto understand what type of data you are storing based on the value assign.

```
x = 5
z = "John"
fruit = "apple"    ---> String
fruit = 10         ---> Integer
```

fruit = Variable name

apple = variable data



- x = 5 stores the integer value 5 under the name x.
- z = "John" stores the text "John" under the name z.
- A variable can later receive a different value.
- Python automatically associates the appropriate type with the assigned value.

Data Types

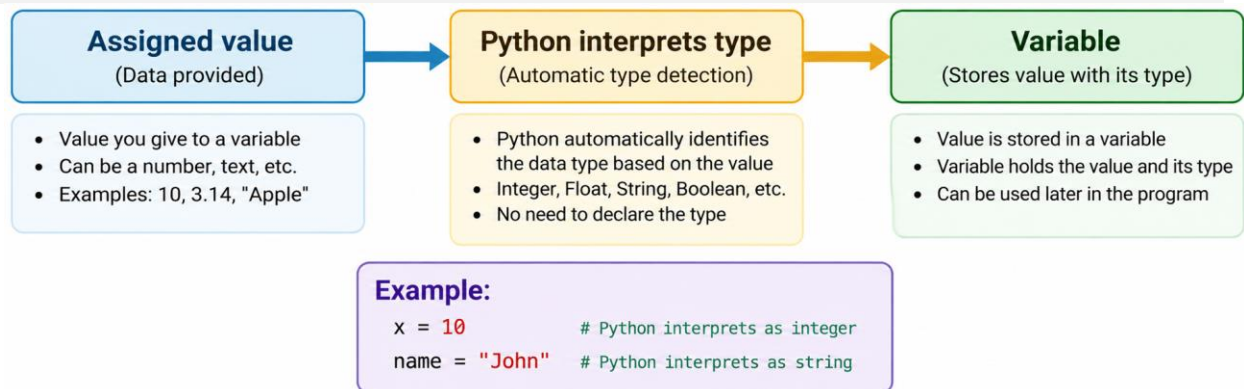
Python data types such as Integer, String, Float and Boolean.

The examples demonstrate that the type depends on the value assigned.

Example	Type	Meaning
fruit = 10	int	Whole number
fruit = "abc"	str	Text
fruit = 10.5	float	Decimal number
is_valid = True	bool	True / False value

```
fruit = 10
print(type(fruit))
# <class "int">
```

```
fruit = "abc"
print(type(fruit))
# <class "str">
```



Why Variables Are Needed

The variables avoid repeating the same value throughout a program.

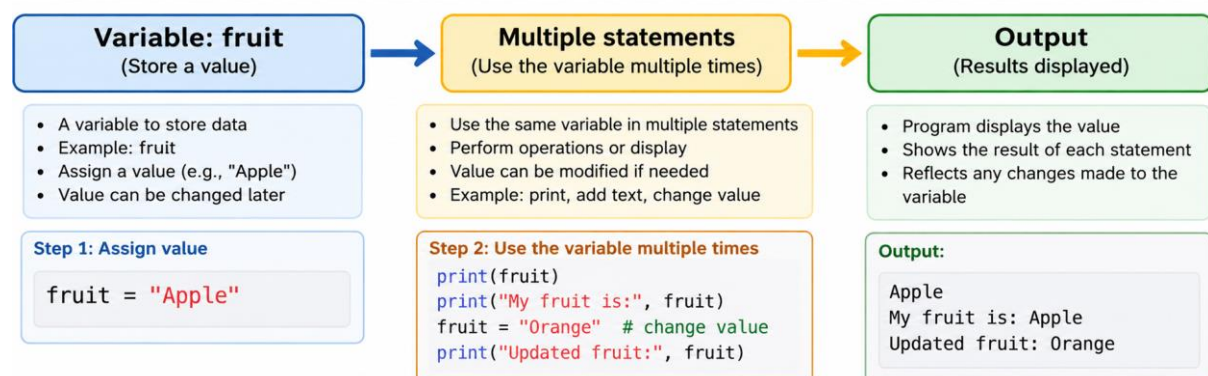
If a value changes, changing the variable once is easier than changing every occurrence.

```
fruit = "apple"

print("fruit name is", fruit)

print("my fruit is", fruit)

fruit = "orange"
```



Constants

A constant as a value intended to **remain unchanged throughout the program**.

Python does not enforce constants with a dedicated keyword; a common convention is to use uppercase names.

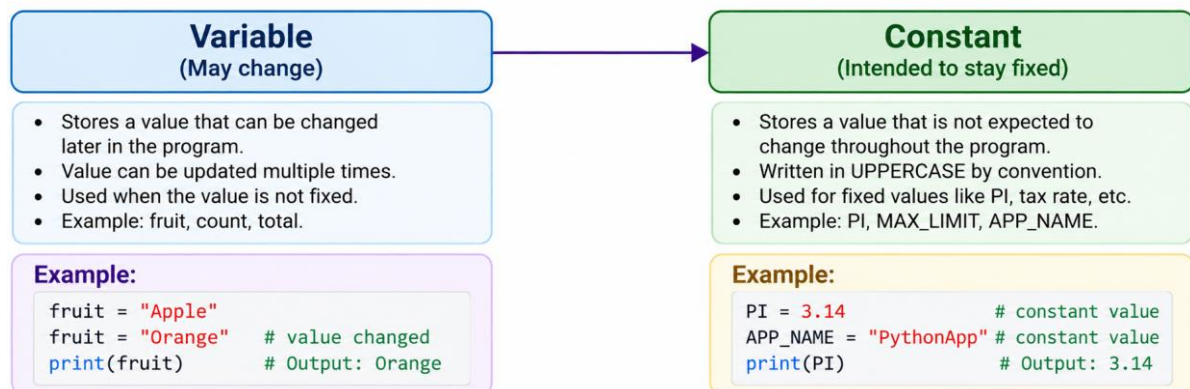
```
PI = 3.14
radius = 5
area = PI * radius * radius
circumference = 2 * PI * radius

HRS_IN_DAY = 24
HRS_IN_MONTH = 30 * HRS_IN_DAY
HRS_IN_YEAR = 365 * HRS_IN_DAY
```

- Improve clarity.
- Reduce accidental changes.
- Make formulas easier to read and maintain.

Variable vs Constant

Aspect	Variable	Constant (convention)
Value	Can change during program execution	Intended to stay the same
Example	radius = 5	PI = 3.14
Purpose	Dynamic program data	Fixed/reference values



Naming Conventions

Help to understand,

Easy to read and Easy to use.

Variable Rules

1. Case Sensitivity (Uppercase and Lowercase)

Example: myVariable=12
MyVariable=13

Stick one style of letter casing and use it consistently throughout your program.

Variable Naming Styles

Camel Case = myVariableName
Pascal Case = MyVariableName
snake case = my_variable_name

Camel Case

myVariableName = "Camel Case"

m - Lowercase

V and N - Uppercase

First letter of the variable name is lowercase while first letter of each subsequent word is capitalized.

Example: totalAmount = 1000

Pascal Case

MyVariableName = "Pascal Case"

M, V & N - Uppercase

First letter of every word & sometimes including the first word capitalized.

Example: CustomerName = "John Due"

TotalAmount = 100

snake case

my_variable_name = "snake Case"

Which separates words with underscores & uses all lowercase letters

Example: user_name = "john Doe"

Max_value = 100

Among all these cases, the snake case is the most commonly used in Python

2. Cannot Contain Spaces

Variable names can't contain spaces

Example: User Name = "john Doe" ----- **Wrong**

userName = "john Doe" ----- **Camel**

UserName = "john Doe" ----- **Pascal**

User_name = "john Doe" ----- **Snake**

3. Can't Start with Digits

Example:

```
2value = 100
Value2 = 100
```

Recommended, the variable names must begin with a letter **a to z** and **A to Z**

Example: a – z, A – Z or (_)

Example:

```
_add = 100
print(_add)
```

4. Can't Contain Special Characters

Example: Percentage % = 92.3 ---- **Wrong**
@, #, %

5. Can't Use Reserved Keywords

Example:

Int, if, def, while

Python use these keywords internal functionality

6. Should Be Descriptive & Meaningful

X = 100 --- **Wrong**
Temp = 200 --- **Wrong**

```
total_amount = 100
user_age = 25
```

Your code easier to understand for others & for yourself when revisiting it after sometime.

Rules demonstrated in the notes

- Python variable names are case-sensitive.
- A variable name cannot contain spaces.
- A variable name should not start with a digit.
- Special characters such as %, @ and # are not valid in ordinary variable names.
- Reserved keywords such as if, def, while and similar Python keywords should not be used as variable names.
- Choose descriptive names so the code is easier for yourself and other developers to understand.

```
user_name = "John Doe"
max_value = 100
# Good: total_amount = 1000
# Avoid: 100value = 10
```

```
# Avoid: user name = "John"
```



Meaningful name

(Descriptive and clear)

- Use names that describe the purpose of the variable.
- Makes the code easy to understand.
- Helps you and others to read the code later.
- Follow naming rules (letters, numbers, underscore).

Example:

```
fruit_name = "Apple"  
total_count = 10
```



Readable code

(Easy to understand)

- Meaningful names make the code self-explanatory.
- Other developers can understand your code quickly.
- Reduces confusion and errors.
- Improves collaboration and debugging.

Example:

```
apple_price = 50  
print(apple_price)  
# Output: 50
```



Easier maintenance

(Simple to modify in future)

- Descriptive names help when you come back to the code later.
- You can easily identify what each variable is used for.
- Makes updates and changes faster.
- Reduces the time and effort in maintenance.

Example:

```
user_name = "John"  
user_name = "Ranjan" # easy to update
```